

Python : Listes, Dictionnaires et Organisation des Données (AoS vs SoA)

Cours d'informatique - CEGEP

10 septembre 2026

1 Manipulations fondamentales des Dictionnaires et Listes

Python propose des structures natives puissantes pour manipuler les associations et les séquences.

- **La fonction `zip()`** : Combine plusieurs listes en un itérable de paires (ou tuples). Utilisée avec `dict()`, elle permet de construire un dictionnaire à partir de listes parallèles :

```
dict(zip(list1, list2))
```

- **Extraction des vues :**

- `dict.keys()` : Retourne la vue de toutes les clés.
- `dict.values()` : Retourne la vue de toutes les valeurs.
- `dict.items()` : Retourne la vue des paires (**clé**, **valeur**).

- **La fonction `enumerate()`** : Permet de parcourir une séquence tout en gardant une trace de l'indice courant, évitant la gestion manuelle de compteurs.

2 Organisation des données : AoS *vs* SoA

La façon dont on structure les listes et les dictionnaires en mémoire impacte directement la performance et la lisibilité du code.

- **AoS (*Array of Structures* / **Tableau de structures**)** : Représenté en Python par une liste de dictionnaires (ex. `['id': 1, 'x': 10, 'id': 2, 'x': 20]`). Idéal pour manipuler des entités complètes de manière orientée objet.
- **SoA (*Structure of Arrays* / **Structure de tableaux**)** : Représenté par un dictionnaire de listes (ex. `'id': [1, 2], 'x': [10, 20]`). Idéal pour le traitement par lot, la vectorisation et l'optimisation du cache matériel (en lien avec le SIMD).

3 Exemple d'utilisation en Python

```
def demonstration_structures():  
    # 1. Utilisation de zip pour creer un dictionnaire  
    noms = ["Alice", "Bob", "Charlie"]  
    scores = [95, 82, 88]  
    registre = dict(zip(noms, scores))  
  
    # 2. Utilisation de .items() et enumerate()  
    print("--- Affichage avec enumerate et .items() ---")  
    for index, (nom, score) in enumerate(registre.items()):  
        print(f"Rang {index} | Etudiant : {nom} | Score : {score}")  
  
    # 3. Structure de donnees SoA (Structure of Arrays)  
    particles_soa = {  
        "id": [1, 2, 3],  
        "x": [0.0, 5.2, -1.3],  
        "velocity": [10.5, 20.0, 15.2]  
    }  
  
    # Mise a jour en lot (simulant un traitement SIMD conceptuel)  
    boost = 1.1  
    for i, v in enumerate(particles_soa["velocity"]):  
        particles_soa["velocity"][i] = v * boost  
  
if __name__ == "__main__":  
    demonstration_structures()
```